SIAM J. SCI. COMPUT.
Vol. 35, No. 2, pp. A603–A622
© 2013 Society for Industrial and Applied Mathematics

Downloaded 04/20/26 to 94.72.173.234 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

AN EXTENSION OF THE DISCONTINUOUS GALERKIN METHOD FOR THE SINGULAR POISSON EQUATION*

FLORIAN KUMMER† AND MARTIN OBERLACK‡

Abstract. We present a numerical method for solving a singular Poisson equation which solution contains jumps and kinks due to a singular right-hand side. Equations of this type may arise, e.g., within the pressure computation of incompressible multiphase flows. The method is an extension to the well-known discontinuous Galerkin (DG) method, being able to represent the jumps and kinks with subcell accuracy. In the proposed method, an ansatz function which already fulfills the jump condition is subtracted from the original problem, thereby reducing it to a standard Poisson equation without a jump. Invoking a technique that we refer to as "patching," the construction of the ansatz function can be limited to a very narrow domain around the jump position, thus making the construction numerically cheap and easy. Under optimal conditions, the method shows a convergence order of $p+1$ for DG polynomial degree p. Still, in the worst case, a convergence order of approximately 2.4 is preserved for DG polynomial degree of 2.

Key words. discontinuous Galerkin, level set, jump operator, multiphase flows, symmetric interior penalty, Poisson equation

AMS subject classifications. 00A72, 00A69, 76M25

DOI. 10.1137/120878586

1. Introduction and problem definition.

Within this paper, we are going to present a numerical method for solving a singular Poisson equation which solution contains jumps and kinks due to a singular right-hand side. Equations of this type may arise, e.g., within the pressure computation of incompressible multiphase flows, as we discuss in section 1.2.

The equation is defined on a polygonal domain $\Omega \subset \mathbb{R}^D$ ($D \in \{2,3\}$), which is partitioned into two disjoint but adjacent subdomains $\mathfrak{A}$ and $\mathfrak{B}$. The Laplace operator is defined everywhere on $\Omega \setminus \mathfrak{I}$, where $\mathfrak{I} = \overline{\mathfrak{A}} \cap \overline{\mathfrak{B}}$ denotes the interface between $\mathfrak{A}$ and $\mathfrak{B}$. We assume that $\mathfrak{I}$ is a $(D-1)$-dimensional manifold which $\mathcal{C}^1$ is almost everywhere. This setup is illustrated in Figure 1.1. At $\mathfrak{I}$, a jump and a kink condition are employed in order to couple the separate domains. We refer to these equations as singular PDEs because the "two-domain formulation" (two separate PDEs for both subdomains and jump conditions for coupling) can be transformed into a "singular-domain formulation" (a single PDE for the whole domain with singular right-hand side, e.g., some delta distribution).

From a numerical point of view, the representation and computation of discontinuous solutions is difficult, especially for higher-order methods. In the best case, only the convergence rate of the scheme degenerates; in the worst case, the method becomes numerically unstable. Hence, there is a strong need for higher-order methods which are able to cope with discontinuities. In the context of Galerkin methods or Petrov–

*Submitted to the journal's Methods and Algorithms for Scientific Computing section May 24, 2012; accepted for publication (in revised form) December 11, 2012; published electronically March 5, 2013. This work was supported by the German Research Foundation (DFG) through the Collaborative Research Center 568 (SFB 568).
http://www.siam.org/journals/sisc/35-2/87858.html
†Chair of Fluid Dynamics, TU Darmstadt, Petersenstraße 30, 64287 Darmstadt, Germany and Graduate School CE, Dolivostraße 15, 64293 Darmstadt, Germany (kummer@fdy.tu-darmstadt.de).
‡Cluster Smart Interfaces, TU Darmstadt, Petersenstraße 30, 64287 Darmstadt, Graduate School CE, Dolivostraße 15, 64293 Darmstadt, Germany (oberlack@fdy-tu-darmstadt.de).

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

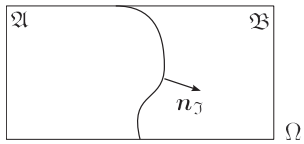

FIG. 1.1. *Illustration of domain partitioning: domain $\mathfrak{A}$, domain $\mathfrak{B}$, and the interface $\mathfrak{I}$. $\Omega = \mathfrak{A} \cup \mathfrak{I} \cup \mathfrak{B}$.*

Galerkin methods, for instance finite element (FEM) or discontinuous Galerkin (DG), it is straightforward to adopt the approximation space in order to represent the desired jump conditions.

In the case of FEM, such an idea was initially proposed by Moës, Dolbow, and Belytschko [19] in order to model solid mechanics problems, which contain cracks and other kinds of discontinuities induced by material defects. This method, referred to as extended finite element (XFEM), is widely applied in fluid dynamics; see, e.g., [18, 17, 9, 12, 29, 5].

In the present paper we focus on an extension of the DG method for a Poisson equation that we will refer to as extended DG (XDG). A symmetric interior penalty method, introduced in 1982 by Arnold [3] and Arnold et al. [2], will be used to discretize the Laplace operator.

The enriched/extended DG approximation space is capable of representing jumps and kinks with subcell accuracy. Ideas similar to this have been proposed and demonstrated by Engwer and co-workers [4, 8, 7].

In contrast, the method that we propose is easier to implement, since no numerical integration over $\mathfrak{I}$ and no explicit reconstruction of $\mathfrak{I}$ is required. Furthermore, the additional degrees of freedom (DOFs) which are introduced by the extension are completely determined by the jump and the kink conditions. Therefore, the solution of the singular Poisson equation is only slightly more expensive than that of a comparable regular problem without jump. Further, the matrix of the linear system that has to be solved is independent of the interface $\mathfrak{I}$ and the symmetric structure of the interior penalty discretization is preserved. This is especially beneficial in the case of a multiphase Navier–Stokes solver (see section 1.2), where the Poisson problem has to be solved multiple times with a different interface position at each time step. However, these advantages come at the price that our method is less general than those proposed in [4, 8, 7], since it is limited to the case of affine jump conditions, i.e., jump conditions of the form $[\![u]\!] = g$. (For the definition of the jump operator $[\![-]\!]$, see (1.1).) Linear-affine jump conditions like $[\![\alpha u]\!] = g$, with $[\![\alpha]\!] \neq 0$, can be treated by an iterative procedure (see section 4), but require multiple solutions of purely affine problems. In this respect, the work done by Engwer and co-workers [4, 8, 7] is a more general approach capable of solving linear-affine jump conditions directly.

1.1. Problem definition.

Jump operator and restricted differential operators. For a function $f \in \mathcal{C}^1(\Omega \setminus \mathfrak{I})$ which is smooth everywhere except on $\mathfrak{I}$ and $\boldsymbol{x} \in \mathfrak{I}$ we define the jump operator

$$[\![f]\!](\boldsymbol{x}) := \lim_{\alpha \searrow 0} \left(f(\boldsymbol{x} + \alpha \boldsymbol{n}_{\mathfrak{I}}) - f(\boldsymbol{x} - \alpha \boldsymbol{n}_{\mathfrak{I}}) \right), \tag{1.1}$$

where $\boldsymbol{n}_{\mathfrak{I}} : \mathfrak{I} \to \mathbb{R}^D$ denotes the normal field on $\mathfrak{I}$. It is oriented in a direction that "points from $\mathfrak{A}$ to $\mathfrak{B}$," i.e., $\boldsymbol{n}_{\mathfrak{I}} = \boldsymbol{n}_{\partial\mathfrak{A}} = -\boldsymbol{n}_{\partial\mathfrak{B}}$ for the outer normal fields $\boldsymbol{n}_{\partial\mathfrak{A}}$ and $\boldsymbol{n}_{\partial\mathfrak{B}}$ of $\mathfrak{A}$ and $\mathfrak{B}$, respectively.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 04/20/26 to 94.72.173.234 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Downloaded 04/20/26 to 94.72.173.234 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

For the notation of PDEs with jumps, or other types of singularities, we found it useful to append the domain of the differential operator to its symbol, e.g., for writing "Laplace everywhere, except the interface." In general, for an open set $X \subset \Omega$ and a function $f \in H^n(\Omega)$ we define, for a differential operator ∂^α,

$$\partial^\alpha|_X f := \begin{cases} \partial^\alpha(f|_X) & \text{in } X, \\ 0 & \text{in } \Omega \setminus X, \end{cases}$$

the restriction of ∂^α onto X. For instance, let H be the Heaviside function; then $\frac{\partial}{\partial x}|_{\mathbb{R}\setminus\{0\}} H(x) = 0$, while $\frac{\partial}{\partial x} H(x)$ is not defined at $x = 0$, at least not in the classical sense.

The singular Poisson equation. The general form of the problem that will be investigated within this work reads as

$$\begin{cases} -\Delta|_{\Omega\setminus\mathfrak{I}} u = f & \text{in } \Omega, \\ [\![u]\!] = g_1 & \text{on } \mathfrak{I}, \\ [\![\nabla u \cdot \boldsymbol{n}_{\mathfrak{I}}]\!] = g_2 & \text{on } \mathfrak{I}, \\ u = g_{\text{Diri}} & \text{on } \Gamma_{\text{Diri}}, \\ \nabla u \cdot \boldsymbol{n}_{\partial\Omega} = g_{\text{Neu}} & \text{on } \Gamma_{\text{Neu}}, \end{cases} \tag{1.2}$$

with the disjoint decomposition $\partial\Omega = \Gamma_{\text{Diri}} \cup \Gamma_{\text{Neu}}$.

The "two-domain" formulation of the problem given above could be reexpressed in a distributional sense, i.e.,

$$-\Delta u = f - \delta_{\mathfrak{I}} g_1 - \operatorname{div}(\delta_{\mathfrak{I}} g_2 \boldsymbol{n}_{\mathfrak{I}}) \text{ in } \Omega, \tag{1.3}$$

where $\delta_{\mathfrak{I}}$ denotes the Dirac δ distribution on $\mathfrak{I}$. Although the equivalence of problems (1.2) and (1.3) (plus boundary conditions) is beyond the scope of this paper; we mention it to motivate the name "singular equation."

1.2. Connection to multiphase Navier–Stokes problems. Our main motivation for the presented method is the treatment of immiscible two-phase and multiphase flows. There, the sets $\mathfrak{A}$, $\mathfrak{B}$, and $\mathfrak{I}$ are time dependent and the density of the fluids is piecewise constant, i.e.,

$$\rho(t, \boldsymbol{x}) = \begin{cases} \rho_{\mathfrak{A}} & \text{if } \boldsymbol{x} \in \mathfrak{A}(t), \\ \rho_{\mathfrak{B}} & \text{if } \boldsymbol{x} \in \mathfrak{B}(t) \end{cases}$$

and the domains are referred to as "fluid phases"; e.g., domain $\mathfrak{A}(t)$ denotes water and $\mathfrak{B}(t)$ denotes oil or air. If physical properties, such as density and viscosity, are discontinuous at the interface between two immiscible fluids, the pressure and velocity field will also contain discontinuities. For material interfaces, when there is no mass flux across the interface, the velocity is always continuous in the normal direction, but it includes a kink (for some velocity component w, $[\![\nabla w \cdot \boldsymbol{n}_{\mathfrak{I}}]\!] \neq 0$). Furthermore, the pressure field incorporates a kink at the interface too, due to density differences, and a jump if surface tension is considered. For nonmaterial interfaces, both the velocity and pressure fields will include jumps, even in the absence of surface tension. A comprehensive overview on the modeling of multiphase flows is given, e.g., by Hutter and Jöhnk [13] or Wang and Oberlack [30].

A solver for problem (1.2) is a key ingredient of a multiphase Navier–Stokes solver, based on the projection approach [1], which will be an issue of future work. In order to motivate the present work, we anticipate the projection step of the multiphase

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 04/20/26 to 94.72.173.234 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Navier–Stokes solver for a time step $\triangle t$. For an intermediate velocity $\boldsymbol{w}^*$, which is the result of the predictor step, one has to solve the Poisson equation

$$\Delta|_{\Omega\setminus\mathfrak{I}}\, p = \operatorname{div}\left(\frac{1}{\triangle t}\rho^1\boldsymbol{w}^*\right) + \frac{1}{\triangle t^2}(\rho^1-\rho^0)$$

with jump conditions

$$[\![\nabla p\cdot\boldsymbol{n}_{\mathfrak{I}}]\!] = [\![\frac{-1}{\Delta t}\rho^1\boldsymbol{w}^*\boldsymbol{n}_{\mathfrak{I}}]\!] \quad \text{and} \quad [\![p]\!] = \sigma\kappa,$$

where σ is the surface tension, κ is the mean curvature of $\mathfrak{I}$, and ρ^1, ρ^0 denote density for the new and the old time step, respectively, in order to compute the pressure p. Then, a velocity field $\boldsymbol{w}^1$, which is divergence free in both of the subdomains can be obtained as $\boldsymbol{w}^1 = \boldsymbol{w}^* - \frac{1}{\triangle t}\frac{1}{\rho^1}\,\nabla|_{\Omega\setminus\mathfrak{I}}\, p$.

2. The construction of extended DG. The numerical treatment of singular equations requires two essential ingredients. At first, the position of the singularity, i.e., the position of the interface $\mathfrak{I}$, must be described, which is done by the well-known level set method; see section 2.2. Second, the approximation space is adapted to represent the singularities, or discontinuities, of the solution u; see section 2.3. Before going into details, the "standard" DG method and the interior penalty discretization of the Laplace operator will be introduced. It should be mentioned that our method is not limited to the symmetric interior penalty discretization of the Poisson equation and could be easily adapted to any of the usual DG–Poisson discretizations. For an overview we refer to Arnold et al. [2].

2.1. Formal definition of classical discontinuous Galerkin method.

Grid and DG approximation. For the polygonal computational domain $\Omega\subset\mathbb{R}^D$, with $D\in\{2,3\}$, the numerical grid needs to be defined. The usual inner product in the $L^2(\Omega)$ sense is notated as $\langle f,g\rangle = \int_\Omega fg\,\mathrm{d}\boldsymbol{x}$.

Given is a set $\mathfrak{K}=\{K_1,\dots,K_J\}$ of cells $K_j\subset\Omega$, which fulfil the usual requirements for a numerical grid, i.e., they do not overlap and $\overline{\Omega}=\bigcup_{K\in\mathfrak{K}}\overline{K}$. For each cell K_j, there is a diffeomorphism T_j such that K_j can be expressed as the image of a reference element $K_{\mathrm{ref}}\subset\mathbb{R}^D$, which is usually a triangle or a quadrilateral (for $D=2$), or a tetrahedron or a hexahedron (for $D=3$), i.e. $K_j=T_j(K_{\mathrm{ref}})$.

The *DG space* of degree p, is defined as

$$\mathbb{P}_p(\mathfrak{K}) := \left\{f\in L^2(\Omega);\ \forall\, K\in\mathfrak{K} : f|_K \text{ is polynomial and } \deg\left(f|_K\right)\le p\right\},$$

where $\deg(f)$ denotes the degree of polynomial f. We further define the projector $Proj_p$ onto $\mathbb{P}_p(\mathfrak{K})$,

$$Proj_p : L^2(\Omega)\to\mathbb{P}_p(\mathfrak{K})$$

given by the characteristic property

$$\left\langle Proj_p(f), v\right\rangle = \langle f,v\rangle \qquad \forall v\in\mathbb{P}_p(\mathfrak{K}).$$

The interior penalty method. Since the present contribution solely focuses on the Poisson equation, we will only specify the DG discretization of the Laplace operator given by the so-called interior penalty method [3, 2].

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 04/20/26 to 94.72.173.234 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

The "classical" Poisson equation—in contrast to the one with jump (1.2)—is given as

$$\left\{\begin{aligned} -\Delta u &= f && \text{in } \Omega,\\ u &= g_{\text{Diri}} && \text{on } \Gamma_{\text{Diri}},\\ \nabla u \cdot \boldsymbol{n}_{\partial\Omega} &= g_{\text{Neu}} && \text{on } \Gamma_{\text{Neu}}. \end{aligned}\right. \tag{2.1}$$

In order to define the interior penalty method, a few auxiliary definitions are required. At first, all edges of all cells are combined in the set Γ:

$$\Gamma = \bigcup_{K\in\mathfrak{K}} \partial K.$$

Almost everywhere on Γ a normal field $\boldsymbol{n}_\Gamma$ is given. On the domain boundary $\partial\Omega$ it is chosen to match the outer normal field $\boldsymbol{n}_{\partial\Omega}$. Since on interior edges, i.e., on $\Gamma\setminus\partial\Omega$, there is no natural choice for the sign of $\boldsymbol{n}_\Gamma$, one of the two possible directions must be chosen. The bilinear form defined by (2.4) is of course independent of this choice, since a swap of the normal sign also swaps the sign of the jump operator and therefore both swaps cancel out.

The extension of the jump operator $[\![-]\!]$ (see (1.1)) onto $\Gamma\setminus\partial\Omega$ is straightforward: for $\boldsymbol{x}\in\Gamma\setminus\partial\Omega$, with normal $\boldsymbol{n}_\Gamma$, we define:

$$[\![u]\!](\boldsymbol{x}) := \lim_{\xi\searrow 0}\left(u(\boldsymbol{x}+\xi\boldsymbol{n}_\Gamma) - u(\boldsymbol{x}-\xi\boldsymbol{n}_\Gamma)\right). \tag{2.2}$$

Furthermore, the mean value operator is required:

$$\{\{u\}\}(\boldsymbol{x}) := \lim_{\xi\searrow 0}\frac{1}{2}\left(u(\boldsymbol{x}+\xi\boldsymbol{n}_\Gamma) + u(\boldsymbol{x}-\xi\boldsymbol{n}_\Gamma)\right). \tag{2.3}$$

On the boundary $\partial\Omega$ these are defined as

$$[\![u]\!](\boldsymbol{x}) := \lim_{\boldsymbol{y}\to\boldsymbol{x}} u(\boldsymbol{y}) \text{ and } \{\{u\}\}(\boldsymbol{x}) := \lim_{\boldsymbol{y}\to\boldsymbol{x}} u(\boldsymbol{y}).$$

The symmetric interior penalty method (SIP) [3] is given by the bilinear form $a_{\text{SIP}}(u,v)$ for $u,v\in\mathbb{P}_p(\mathfrak{K})$. Below, for the definition of the XDG–Poisson solver, it will be required to treat volume and surface terms separately. Therefore, a decomposition of the usual notation for the SIP form will be given:

$$a_{\text{SIP}}(u,v) := a^{\text{vol}}_{\text{SIP}}(u,v) + a^{\oint}_{\text{SIP}}[\Gamma](u,v) \quad \text{with} \tag{2.4}$$

$$a^{\text{vol}}_{\text{SIP}}(u,v) := -\int_\Omega \nabla_h u\cdot\nabla_h v \; d\boldsymbol{x}, \tag{2.5}$$

$$\begin{aligned} a^{\oint}_{\text{SIP}}[X](u,v) := &-\int_{X\setminus\Gamma_{\text{Neu}}} \mu\,[\![v]\!]\,[\![u]\!] \; d\text{S}\\ &+\int_{X\setminus\Gamma_{\text{Neu}}} \left([\![v]\!]\,\{\{\nabla_h u\}\} + [\![u]\!]\,\{\{\nabla_h v\}\}\right)\cdot\boldsymbol{n}_\Gamma \; d\text{S}, \end{aligned} \tag{2.6}$$

where $\nabla_h := \nabla|_{\Omega\setminus\Gamma}$ denotes the broken gradient. The so-called penalty parameter $\mu\in\mathbb{R}_{>0}$ must be chosen large enough to ensure coercivity of the bilinear form. Yet a larger penalty parameter increases the condition number of the matrix and in turn increases the costs of running an iterative sparse solver. For computing μ, a lower-bound estimation from [28] is used.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 04/20/26 to 94.72.173.234 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

The inhomogeneous Dirichlet and Neumann boundary conditions are represented in the linear form

$$c'[g_{\mathrm{Diri}}, g_{\mathrm{Neu}}](v) := -\int_{\Gamma_{\mathrm{Diri}}} g_{\mathrm{Diri}} \left(\nabla_h v \cdot \boldsymbol{n}_{\partial\Omega} - \mu v\right) \, d\mathrm{S} + \int_{\Gamma_{\mathrm{Neu}}} g_{\mathrm{Neu}} v \, d\mathrm{S}$$

as well as the right-hand side of (1.2) or (2.1):

$$f'(v) := \int_{\Omega} f v \, d\boldsymbol{x}.$$

Using these definitions, the SIP method reads as follows: Find $u \in \mathbb{P}_p(\mathfrak{K})$ so that

$$-a_{\mathrm{SIP}}(u, v) := c'[g_{\mathrm{Diri}}, g_{\mathrm{Neu}}](v) + f'(v) \quad \forall v \in \mathbb{P}_p(\mathfrak{K}). \tag{2.7}$$

2.2. The level set framework. The interface $\mathfrak{I}$ is represented as the zero isosurface of the level set function $\varphi : \Omega \to \mathbb{R}$, which is supposed to be $\mathcal{C}^1$ almost everywhere. Then, the interface

$$\mathfrak{I} = \{\boldsymbol{x} \in \Omega;\ \varphi(\boldsymbol{x}) = 0\}$$

as well as the domains

$$\mathfrak{A} = \{\boldsymbol{x} \in \Omega;\ \varphi(\boldsymbol{x}) < 0\} \text{ and } \mathfrak{B} = \{\boldsymbol{x} \in \Omega;\ \varphi(\boldsymbol{x}) > 0\}$$

are determined by φ. It is further assumed that φ fulfils a signed-distance property, i.e.,

$$|\varphi(\boldsymbol{x})| = \min_{\boldsymbol{x}_0 \in \mathfrak{I}} |\boldsymbol{x}_0 - \boldsymbol{x}|.$$

Under these assumptions, the gradient $\nabla\varphi$ defines an oriented normal field on $\mathfrak{I}$, which points from $\mathfrak{A}$ to $\mathfrak{B}$, i.e., $\boldsymbol{n}_{\mathfrak{I}} = \nabla\varphi$ on $\mathfrak{I}$ because $|\nabla\varphi| = 1$ almost everywhere.

The construction of signed-distance level set fields is indeed a nontrivial problem. However, since numerical level set algorithms can be considered general knowledge, we refer to the textbooks [25, 26, 21] and the works [27, 24] for further reading. For a DG-specific level set implementation we refer to [11].

2.3. Extension of the DG method. For a level set function φ, the set of *cut cells* is defined as:

$$\mathit{Cut}(\varphi) := \{K \in \mathfrak{K};\ \mathrm{Vol}_{D-1}\left(K \cap \mathfrak{I}\right) > 0\}. \tag{2.8}$$

Here, $\mathrm{Vol}_D\left(X\right)$ denotes the D-dimensional measure of $X \subset \Omega$. The XDG space is defined as

$$\begin{aligned} \mathbb{P}_p^X(\varphi, \mathfrak{K}) := \big\{ f \in L^2(\Omega);\ \forall\ K \in \mathfrak{K} : f|_{K\cap\mathfrak{A}}, f|_{K\cap\mathfrak{B}} &\text{ are polynomial,} \\ &\deg\left(f|_{K\cap\mathfrak{A}}\right) \le p \text{ and } \deg\left(f|_{K\cap\mathfrak{B}}\right) \le p \big\}. \end{aligned} \tag{2.9}$$

We further define the continuation mapping in the cut cells,

$$\mathit{Cont}_{\mathcal{S}} : \mathbb{P}_p^X(\varphi, \mathfrak{K}) \to \mathbb{P}_p(\mathfrak{K}),$$

for $\mathcal{S} \in \{\mathfrak{A}, \mathfrak{B}\}$, which continues $u \in \mathbb{P}_p^X(\varphi, \mathfrak{K})$ from "one side" of $\mathfrak{I}$ into the domain of cut cells. Because of the polynomial structure of domain and codomain, $\mathit{Cont}_{\mathcal{S}}$ is

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 04/20/26 to 94.72.173.234 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

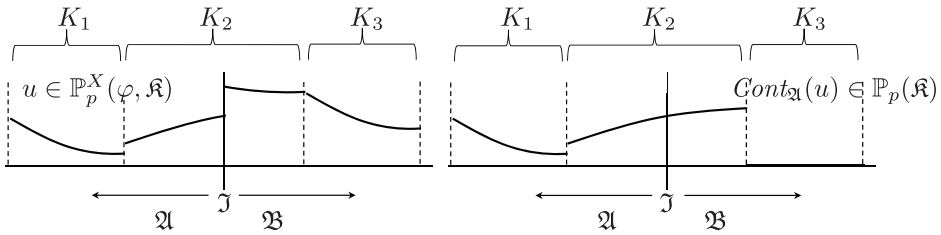

FIG. 2.1. *Continuation mapping. Left:* $u \in \mathbb{P}_p^X(\varphi, \mathfrak{K})$*, right:* $Cont_{\mathfrak{A}}(u) \in \mathbb{P}_p(\mathfrak{K})$*. The example shows a one-dimensional grid consisting of three cells* $\mathfrak{K} = \{K_1, K_2, K_3\}$*. Within* K_1*, since it is fully contained in* $\mathfrak{A}$*, (i.e.,* $\mathfrak{A} \cap K_1 = K_1$*),* $Cont_{\mathfrak{A}}(u)|_{K_1} = u|_{K_1}$*. For* K_3*, which does not intersect with* $\mathfrak{A}$*,* $Cont_{\mathfrak{A}}(u)|_{K_3} = 0$*. Finally for* K_2*, which intersects with both* $\mathfrak{A}$ *and* $\mathfrak{B}$ *there is only one possible continuation in* $\mathbb{P}_p(\mathfrak{K})$ *so that* $Cont_{\mathfrak{A}}(u)|_{K_2 \cap \mathfrak{A}} = u|_{K_2 \cap \mathfrak{A}}$*.*

uniquely determined by the property

$$\begin{cases} Cont_{\mathcal{S}}(u)\mathbf{1}_{\mathcal{S}} = u\mathbf{1}_{\mathcal{S}}, \\ \quad Cont_{\mathcal{S}}(u) = 0 \text{in} \bigcup_{\substack{K \in \mathfrak{K} \\ \mathrm{Vol}_D(K \cap \mathcal{S})=0}} K, \end{cases}$$

where $\mathbf{1}_X$ denotes the characteristic function for some set X. An illustration is given in Figure 2.1.

3. An XDG solution of the singular Poisson equation.

3.1. Jump subtraction and patching.

Outline of ideas on the continuous level. In order to solve the Poisson problem with a jump (1.2) we propose a procedure which contains two ingredients that we refer to as "jump subtraction" and "patching." At first, these procedures should be motivated and explained on the continuous level. Afterwards, the XDG discretization will be given. The procedure is illustrated in Figure 3.1.

We assume to know an ansatz function $u_A \in H^2(\Omega \setminus \mathfrak{I})$ that fulfils the jump condition at the interface, i.e.,

$$\begin{cases} [\![u_A]\!] = g_1 \text{ on } \mathfrak{I}, \\ [\![\nabla u_A \cdot \boldsymbol{n}_{\mathfrak{I}}]\!] = g_2 \text{ on } \mathfrak{I}. \end{cases} \tag{3.1}$$

It is obvious that, in general, the construction of such a function would be numerically expensive. Hence, we limit the support of u_A to a neighborhood K_C of the interface $\mathfrak{I}$ which contains only those cells that are cut by $\mathfrak{I}$, i.e.,

$$K_C := \bigcup_{K \in Cut(\varphi)} \overline{K}$$

and $\mathrm{supp}(u_A) = K_C$. In order to indicate this, we may write $u_A \mathbf{1}_{K_C}$ instead of u_A, i.e., $u_A \mathbf{1}_{K_C} = u_A$. An explicit technique for the construction of u_A will be shown in section 3.2.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

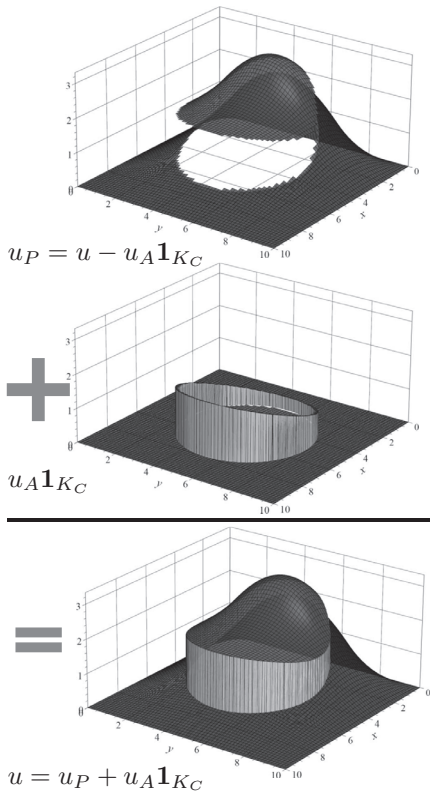

FIG. 3.1. *Illustration of patching, two-dimensional example: The solution u_P of the patched-decomposed equation* (3.5) *shows an "artificial" jump on the boundary of the patching domain, which compensates with the jump in the patched ansatz $u_A \mathbf{1}_{K_C}$. In sum, we obtain the numerical solution of the singular problem.*

Then, instead of solving (1.2), one defines the decomposition

$$u = u_P + u_A = u_P + u_A \mathbf{1}_{K_C}, \tag{3.2}$$

which immediately yields $u_P \in H^2(\Omega \setminus \partial K_C)$, and solves the equation

$$\begin{cases} -\Delta|_{\Omega\setminus\partial K_C} u_P = f + \Delta|_{\Omega\setminus\mathfrak{I}\setminus\partial K_C} u_A & \text{in } \Omega, \\ [\![u_P + u_A]\!] = 0 & \text{on } \partial K_C, \\ [\![(\nabla u_P + \nabla u_A)\cdot \boldsymbol{n}_{\partial K_C}]\!] = 0 & \text{on } \partial K_C, \\ u_P = g_{\text{Diri}} - u_A & \text{on } \Gamma_{\text{Diri}}, \\ \nabla u_P \cdot \boldsymbol{n}_{\partial\Omega} = g_{\text{Neu}} - \nabla|_{\Omega\setminus\Gamma\setminus\mathfrak{I}} u_A \cdot \boldsymbol{n}_{\partial\Omega} & \text{on } \Gamma_{\text{Neu}}, \end{cases} \tag{3.3}$$

where *jump subtraction* is performed within K_C, while the original equation is solved on $\Omega \setminus K_C$. These two domains are *patched* together. Another point of view may be that the jump has been moved from $\mathfrak{I}$ to the cell boundaries, and jump conditions on the interface have been replaced with jump conditions on ∂K_C. Since $\partial K_C \subset \Gamma$, no integration on $\mathfrak{I}$ is necessary in order to construct the weak form of (3.3).

The patching-correction form. Within this paragraph it will be demonstrated how the "artificial" jump in u_P, at ∂K_C, can be compensated for by the form

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 04/20/26 to 94.72.173.234 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Downloaded 04/20/26 to 94.72.173.234 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

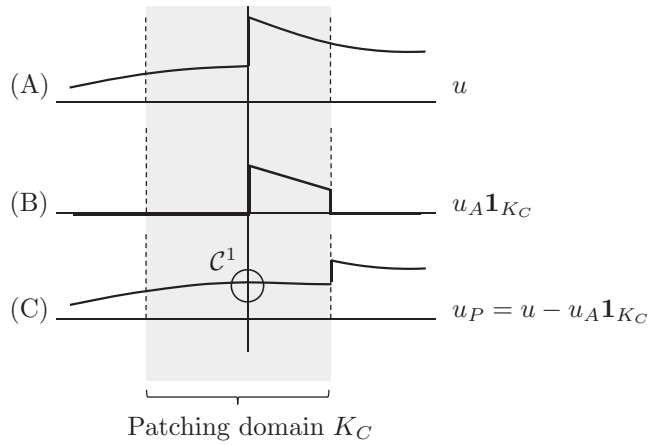

FIG. 3.2. *Illustration of patching, one-dimensional example. One may assume that the solution u of some singular problem is given as sketched in* (A). *When the patched ansatz $u_A\mathbf{1}_{K_C}$ (see* (B)*)—which is equal to zero outside the patching domain K_C—is subtracted from u, two facts are observed:* (i) *jump and kink at $\mathfrak{I}$ are removed, i.e., u_P and its first derivative are continuous;* (ii) *another artificial jump in u_P arises which is located exactly at a cell boundary* (C). *The idea of patching is to correct this jump by the patching-correction form $-a^{\oint}_{SIP}[\partial K_C](u_A\mathbf{1}_{K_C},-)$. In this example, K_C consists of just one cell.*

$a^{\oint}_{\mathrm{SIP}}[\partial K_C](-,-)$. Note that due to the decomposition, $u_P \in \mathbb{P}_p(\mathfrak{K})$ and

$$\begin{aligned}
&a_{\mathrm{SIP}}(u_P,v) - a^{\oint}_{\mathrm{SIP}}[\partial K_C](u_A\mathbf{1}_{K_C},v)\\
&\quad= -\int_\Omega \nabla_h u_P \nabla_h v \; d\boldsymbol{x} - \int_{\Gamma\setminus\Gamma_{\mathrm{Neu}}} \mu\,[\![v]\!]\,\underbrace{[\![u_P + u_A\mathbf{1}_{K_C}]\!]}_{=[\![u]\!]}\; d\mathrm{S}\\
&\qquad+ \int_{\Gamma\setminus\Gamma_{\mathrm{Neu}}} ([\![v]\!]\,\underbrace{\{\{\nabla_h u_P + \nabla_h(u_A\mathbf{1}_{K_C})\}\}}_{=\{\{\nabla_h u\}\}} + \underbrace{[\![u_P + u_A\mathbf{1}_{K_C}]\!]}_{=[\![u]\!]}\,\{\{\nabla_h v\}\})\cdot \boldsymbol{n}_\Gamma \; d\mathrm{S}\\
&\quad= a^{\mathrm{vol}}_{\mathrm{SIP}}(u_P,v) + a^{\oint}_{\mathrm{SIP}}[\partial K_C](u,v),
\end{aligned}$$

as long as equation (3.2) holds. Due to the above identity, the sum

$$a_{\mathrm{SIP}}(u_P,v) - a^{\oint}_{\mathrm{SIP}}[\partial K_C](u_A\mathbf{1}_{K_C},v)$$

is an approximation of $\Delta|_{\Omega\setminus\partial K_C}\, u_P$, i.e.,

$$a_{\mathrm{SIP}}(u_P,v) - a^{\oint}_{\mathrm{SIP}}[\partial K_C](u_A\mathbf{1}_{K_C},v) \approx \langle \Delta|_{\Omega\setminus\partial K_C}\, u_P, v\rangle \quad \forall v \in \mathbb{P}_p(\mathfrak{K})$$

or, in other words, $-a^{\oint}_{\mathrm{SIP}}[\partial K_C](-,-)$ cancels out the penalization of the "artificial" jump of u_P (see Figure 3.2) that arises on ∂K_C and is therefore called a *patching-correction* form.

Restricted evaluation of the Laplacian. It is still an issue to approximate $\Delta|_{\Omega\setminus\mathfrak{I}} u_A$ on the patching domain K_C. Since $u_A = u_A\mathbf{1}_{K_C}$ in general has a rather significant jump at ∂K_C, $a_{\mathrm{SIP}}(u_A,-)$ would be a bad approximation of the Laplacian of u_A. For this reason, we define

$$a_{\mathrm{SIP,rst}}(w,v) := a_{\mathrm{SIP}}(w\mathbf{1}_{K_C},v) - a^{\oint}_{\mathrm{SIP}}[\partial K_C](w\mathbf{1}_{K_C},v) \text{ for } w,v \in \mathbb{P}_p(\mathfrak{K}).$$

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 04/20/26 to 94.72.173.234 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Since the penalty terms (boundary integrals) of $a_{\mathrm{SIP,rst}}$ are zero at ∂K_C, it is an approximation of $\mathbf{1}_{K_C}\,\Delta|_{\Omega\setminus\partial K_C}\,w$, i.e.,

$$a_{\mathrm{SIP,rst}}(w,v) \approx \left\langle \mathbf{1}_{K_C}\Delta|_{\Omega\setminus\partial K_C}\,w, v\right\rangle \text{ for } w,v\in\mathbb{P}_p(\mathfrak{K}). \tag{3.4}$$

XDG scheme for a Poisson equation with an additive jump. At this point we are able to formulate the patched-decomposition scheme for the Poisson equation as it is illustrated in Figure 3.1, which is the central result of this section.

An ansatz function $u_A\in\mathbb{P}_p^X(\varphi,\mathfrak{K})$ which fulfills the jump conditions (3.1) is given. Using this ansatz, one has to find $u_P\in\mathbb{P}_p(\mathfrak{K})$, such that

$$\begin{aligned}-a_{\mathrm{SIP}}(u_P,v) = &-a_{\mathrm{SIP}}^{\oint}[\partial K_C](u_A\mathbf{1}_{K_C},v)\\ &+c'[g_{\mathrm{Diri}}-u_A|_{\Gamma_{\mathrm{Diri}}},g_{\mathrm{Neu}}-(\nabla_h u_A)|_{\Gamma_{\mathrm{Neu}}}](v)+f'(v)\\ &+\sum_{\mathcal{S}\in\{\mathfrak{A},\mathfrak{B}\}} a_{\mathrm{SIP,rst}}(Cont_{\mathcal{S}}(u_A),Proj_p(\mathbf{1}_{K_C\cap\mathcal{S}}v)) \qquad \forall v\in\mathbb{P}_p(\mathfrak{K}).\end{aligned} \tag{3.5}$$

The numerical solution $u\in\mathbb{P}_p^X(\varphi,\mathfrak{K})$ of the discontinuous Poisson problem with jump is given as $u=u_P+u_A$. We want to point out that the method does not require integration over the surface $\mathfrak{I}$.

Rationale. The Poisson equation is given as $-\Delta|_{\Omega\setminus\mathfrak{I}}u=f$; patching and jump subtraction yield

$$-\Delta|_{\Omega\setminus\partial K_C}u_P = f+\mathbf{1}_{K_C}\Delta|_{\Omega\setminus\mathfrak{I}}u_A. \tag{3.6}$$

The DG approximation of (3.6) is obtained as

$$-a_{\mathrm{SIP}}(u_P,v)+a_{\mathrm{SIP}}^{\oint}[\partial K_C](u_A\mathbf{1}_{K_C},v) = c'[\dots](v)+f'(v)+\left\langle\Delta|_{\Omega\setminus\mathfrak{I}\setminus\partial K_C}u_A,v\right\rangle.$$

Considering

$$\left\langle\Delta|_{\Omega\setminus\mathfrak{I}\setminus\partial K_C}u_A,v\right\rangle = \sum_{\mathcal{S}\in\{\mathfrak{A},\mathfrak{B}\}}\left\langle\mathbf{1}_{K_C\cap\mathcal{S}}\Delta|_{\Omega\setminus\partial K_C}Cont_{\mathcal{S}}(u_A),v\right\rangle,$$

it remains to show that $\langle\mathbf{1}_{\mathcal{S}}\Delta|_{\Omega\setminus\partial K_C}w,v\rangle\approx a_{\mathrm{SIP,rst}}(w,Proj_p(\mathbf{1}_{K_C\cap\mathcal{S}}v))$, with $w:=Cont_{\mathcal{S}}(u_A)\in\mathbb{P}_p(\mathfrak{K})$. Let $(\phi_1,\dots\phi_{JN_p})$ be an orthonormal basis of $\mathbb{P}_p(\mathfrak{K})$, i.e., $\langle\phi_k,\phi_l\rangle=\delta_{k,l}$, with J being the number of cells ($J=\#\mathfrak{K}$) and JN_p being the vector-space dimension of $\mathbb{P}_p(\mathfrak{K})$. The operator $L_\Delta:\mathbb{P}_p(\mathfrak{K})\to\mathbb{P}_p(\mathfrak{K})$, which corresponds to $a_{\mathrm{SIP,rst}}$, can then be represented as

$$L_\Delta(w) := \sum_k \phi_k a_{\mathrm{SIP,rst}}(w,\phi_k), \tag{3.7}$$

i.e., $L_\Delta(w)$ is an interior penalty approximation of $\mathbf{1}_{K_C}\Delta|_{\Omega\setminus\partial K_C}w$ for $w\in\mathbb{P}_p(\mathfrak{K})$. Using this, one can derive:

$$\begin{aligned}\left\langle\mathbf{1}_{K_C\cap\mathcal{S}}\Delta|_{\Omega\setminus\partial K_C}w,v\right\rangle &= \left\langle\mathbf{1}_{K_C}\Delta|_{\Omega\setminus\partial K_C}w,\mathbf{1}_{K_C\cap\mathcal{S}}v\right\rangle\\ &\approx \langle L_\Delta(w),\mathbf{1}_{\mathcal{S}}v\rangle\\ &= \sum_k a_{\mathrm{SIP,rst}}(w,\phi_k)\,\langle\phi_k,\mathbf{1}_{K_C\cap\mathcal{S}}v\rangle\\ &= a_{\mathrm{SIP,rst}}(w,\underbrace{\sum_k\phi_k\,\langle\phi_k,\mathbf{1}_{K_C\cap\mathcal{S}}v\rangle}_{=Proj_p(\mathbf{1}_{K_C\cap\mathcal{S}}v)}). \qquad \square\end{aligned}$$

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 04/20/26 to 94.72.173.234 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Note on implementation: Detection of cut cells. A straightforward way to numerically detect cut cells is to evaluate the level set φ at certain points within each cell and test if the sign of φ changes within one cell. Problems may occur if the zero set is located exactly on the edge between two cells. In such a case, it may depend on properties such as numerical round-off errors whether both, only one, or no cell is detected. A simple but effective way to overcome this issue is to choose some of the test vertices *slightly outside* of the corresponding cell, e.g., so that the convex hull of the test vertices is slightly larger (in our experiments, a scaling factor of 1.01 was used) and located symmetrically around the cell that should be tested. By doing so, both neighboring cells will always be detected if the zero set is aligned with their common edge.

3.2. Construction of the ansatz function. Given are $g_1, g_2 \in L^2(\mathfrak{I}) \cap \mathcal{C}^1(\mathfrak{I})$. It is assumed that the curvature of $\mathfrak{I}$ is bounded, which is the case if $\Delta\varphi$ is bounded. An ansatz function u_A, satisfying

$$[\![u_A]\!] = g_1 \text{ on } \mathfrak{I}, \tag{3.8}$$

$$[\![\nabla u_A \cdot \boldsymbol{n}_{\mathfrak{I}}]\!] = g_2 \text{ on } \mathfrak{I}, \tag{3.9}$$

is given in a neighborhood of $\mathfrak{I}$ by

$$u_A := \begin{cases} 0 & \text{for } \boldsymbol{x} \in \mathfrak{A}, \\ \varphi(\boldsymbol{x})g_2(\boldsymbol{x}_0) + g_1(\boldsymbol{x}_0) & \text{for } \boldsymbol{x} \in \mathfrak{B}, \end{cases}$$

with $\boldsymbol{x}_0 = \boldsymbol{x} - \varphi(\boldsymbol{x}) \cdot (\nabla\varphi)(\boldsymbol{x})$ for a signed-distance level set function φ.

Proof. Only considering side $\mathfrak{B}$, by using an infinitesimal difference, one gets $(\nabla u_A)(\boldsymbol{x}_0)\cdot\boldsymbol{n}_{\mathfrak{I}} = \lim_{\epsilon\searrow 0} \frac{1}{\epsilon}\left(u_A(\boldsymbol{x}_0 + \epsilon\boldsymbol{n}_{\mathfrak{I}}) - u_A(\boldsymbol{x}_0)\right) = \lim_{\epsilon\searrow 0} \frac{1}{\epsilon}\left(\varphi(\boldsymbol{x}_0 + \epsilon\boldsymbol{n}_{\mathfrak{I}})g_2(\boldsymbol{x}_0)\right)$. Because of the signed-distance property, we have $\varphi(\boldsymbol{x}_0 + \epsilon\boldsymbol{n}_{\mathfrak{I}}) = \epsilon$, and this yields (3.9). Equation (3.8) is obvious. ∎

The DG projection of u_A can be computed by the following algorithm within each cell $K \in K_C$:

1. Choose a quadrature rule with nodes $\boldsymbol{x}_1, \dots \boldsymbol{x}_L$ and weights $\alpha_1, \dots, \alpha_L$. For a DG polynomial degree of p (of u_A), we propose to use a quadrature rule of at least order $2p$. Usage of higher precision does not seem to be beneficial, as will later be shown in Figure 3.7.
2. Compute the plumb line on $\mathfrak{I}$ through each $\boldsymbol{x}_i$, i.e., $\boldsymbol{x}_{0,i} := \boldsymbol{x}_i - \nabla\varphi(\boldsymbol{x}_i) \cdot \varphi(\boldsymbol{x}_i)$. (Due to the signed-distance property, a line through $\boldsymbol{x}_{0,i}$ and $\boldsymbol{x}_i$ is perpendicular on $\mathfrak{I}$.)
3. Evaluate g_1 and g_2 at the nodes $\boldsymbol{x}_{0,i}$, which lie approximately at the interface.
4. Compute (an approximation to) the DG projection of u_A with the use of the quadrature rule: $\langle Proj_p(u_A), \phi\rangle \approx \sum_{i=1}^{L} \alpha_i(\varphi(\boldsymbol{x}_i)g_2(\boldsymbol{x}_{0,i}) + g_1(\boldsymbol{x}_{0,i}))\phi(\boldsymbol{x}_i)$.

3.3. Numerical results and discussion.

Convergence studies. Within the domain $\Omega = (-2, 2)^2$ two kinds of jumps have been investigated, namely,

- trivial jump: $[\![u]\!] = 1$ and $[\![\boldsymbol{n}_{\mathfrak{I}} \cdot \nabla u]\!] = 0$;
- crown jump: $[\![u]\!] = 2 + 0.7\sin(6\ \theta)$ and $[\![\boldsymbol{n}_{\mathfrak{I}} \cdot \nabla u]\!] = -5\sin(6\ \theta)$, where the angle $\theta = \tan^{-1}(y/x)$;

as well as two different right-hand sides (RHSs),

- smooth RHS: $f(\boldsymbol{x}) = 5\exp(-|\boldsymbol{x}|^2)$;
- discontinuous RHS: $f(\boldsymbol{x}) = \begin{cases} 0 & \text{in } \mathfrak{A}, \\ 0.2\exp(-0.2|\boldsymbol{x}|^2) & \text{in } \mathfrak{B}. \end{cases}$

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

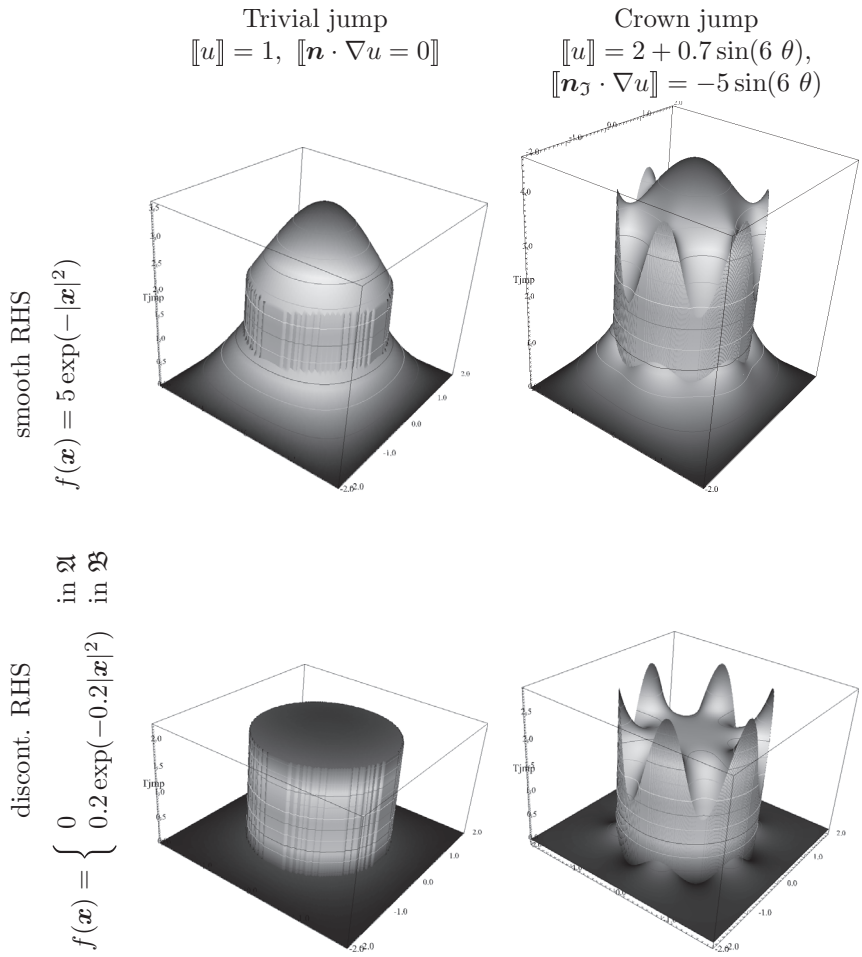

FIG. 3.3. *Numerical solution of the examples used for convergence measurements, on a* 64×64 *equidistant grid with DG polynomial degree* 3.

The interface $\mathfrak{I}$ is a circle of radius 1.3 around the origin, $\mathfrak{A}$ is the exterior and $\mathfrak{B}$ is the interior of this circle, and the corresponding signed-distance level set function $\varphi(\boldsymbol{x}) = 1.3 - |\boldsymbol{x}|_2$ is projected onto $DG_5(\mathfrak{K})$. Numerical solutions for all four combinations are shown in Figure 3.3. DG polynomial degrees 1, 2, 3, and 4 were investigated, on grids with $8 \times 8, 16 \times 16, \ldots, 512 \times 512$ equidistant cells. For polynomial degree 4, only the first 6 of those grids could be investigated due to memory restrictions.

Computation of errors. Errors were investigated in the $L^2(\Omega)$-norm, in the $L^2(\mathfrak{I})$-norm, as well as in the "sip+"-norm,

$$\|u\|_{\text{sip+}} := \left(\|u\|_{\text{sip}}^2 + \|\,[\![u]\!]\,\|_{L^2(\mathfrak{I})}^2\right)^{1/2}, \tag{3.10}$$

which is composed from the well-known "sip"-norm (see, e.g., [6]),

$$\|u\|_{\text{sip}} := \left(\|\nabla_h u\|_{L^2(\Omega)^D}^2 + \left\|\frac{[\![u]\!]}{h_F}\right\|_{L^2(\Gamma)}^2\right)^{1/2}, \tag{3.11}$$

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 04/20/26 to 94.72.173.234 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

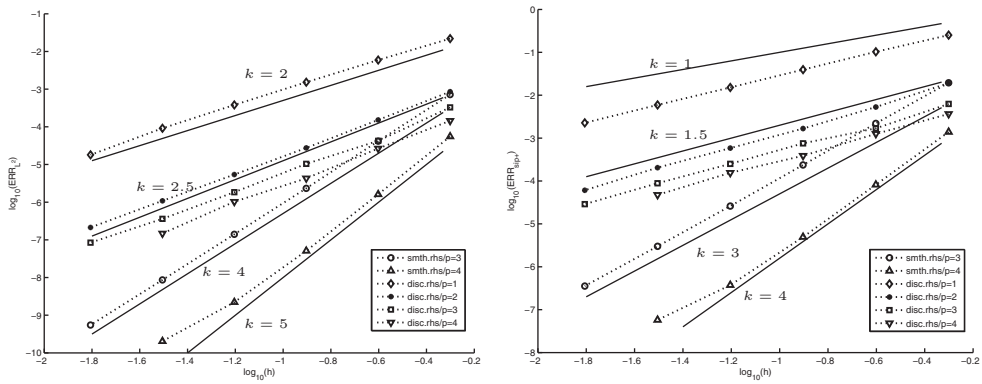

FIG. 3.4. *Convergence plot for trivial jump. The measurements for DG degree 4 and smooth RHS show degeneration for the last two measurements. The reason may be that the finest solution is not precise enough, e.g., due to insufficient sparse-solver accuracy.*

and the L^2-norm of $[\![u]\!]$ on the interface. The piecewise constant h_F denotes the diameter of face $F \subset \Gamma$. The solution on the finest grid was assumed to be the exact solution.

In order to compare solutions, the coarse solutions are injected into the finer grid. This is indeed not trivial: assume $\mathfrak{K}_{ex}$ denotes the finest grid and $\mathfrak{K}_n$ some coarser nested grid, i.e., $\mathbb{P}_p(\mathfrak{K}_n) \subset \mathbb{P}_p(\mathfrak{K}_{ex})$. Since the level set functions $\varphi_n \in \mathbb{P}_p(\mathfrak{K}_n)$ and $\varphi_{ex} \in \mathbb{P}_p(\mathfrak{K}_{ex})$ on these different grid levels are *not* necessarily equal, the XDG spaces are not nested, i.e., $\mathbb{P}_p^X(\varphi_n, \mathfrak{K}_n) \not\subset \mathbb{P}_p^X(\varphi_{ex}, \mathfrak{K}_{ex})$. Hence, we compute the error in some norm $\|-\|_*$ between numerical solutions $u_n \in \mathbb{P}_p^X(\varphi_n, \mathfrak{K}_n)$ and $u_{ex} \in \varphi_{ex} \in \mathbb{P}_p(\mathfrak{K}_{ex})$ as

$$\mathrm{ERR}_*(n) = \|u_{ex} - (Cont_{\mathfrak{A}_1}(u_n)\mathbf{1}_{\mathfrak{A}_{ex}} + Cont_{\mathfrak{B}_1}(u_n)\mathbf{1}_{\mathfrak{B}_{ex}})\|_*\,,$$

where $\mathfrak{A}_1$, $\mathfrak{B}_1$, $\mathfrak{A}_{ex}$, $\mathfrak{B}_{ex}$ denote the positive/negative regions with respect to φ_n and φ_{ex}.

Results. For classical problems with a sufficiently smooth exact solution $u_{ex} \in H^{p+1}(\Omega)$, the error of the numerical solution $u \in \mathbb{P}_p(\mathfrak{K})$ is known to be

$$\|u_{ex} - u\|_2 \le C_1 \cdot h^{p+1}, \text{ respectively, } \|u_{ex} - u\|_{\mathrm{sip}} \le C_2 \cdot h^p,$$

with h, as usual, being a characteristic length of the grid, see, e.g., [6]. Since the solution for the trivial jump is $u_N + \mathbf{1}_{\mathfrak{B}}$, where u_N is the solution for the problem without jump (i.e., $\Delta u_N = f$, $[\![u_N]\!] = [\![\boldsymbol{n}_{\mathfrak{I}} \cdot \nabla u_N]\!] = 0$), one would expect the convergence result to hold also for the trivial jump. For a discontinuous RHS the convergence rate stagnates around 2.4 (in L^2), respectively, 1.6 (in sip+) and DG polynomial degrees greater than 2 yield no improvement in convergence order, as is evident from Figure 3.4 as well as Tables 3.1 and 3.2. This is not surprising, since already for a SIP method without any jump one would expect no more than a convergence order of 2 in the L^2-norm, respectively, 1 in sip-norm, if the exact solution u_{ex} is only a member of $H^2(\Omega)$ but not of $H^3(\Omega)$.

For the "crown jump," the situation seems to be more complex (see Figure 3.5).

- By comparing Figure 3.4 versus Figure 3.5, or, alternatively, the left versus the right-hand side of Tables 3.1 and 3.2, one can see that the error constant is between two and four magnitudes higher than for the trivial jump.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 04/20/26 to 94.72.173.234 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Downloaded 04/20/26 to 94.72.173.234 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

TABLE 3.1
Convergence rates from numerical experiments, in the L^2-norm.

DG degree	Trivial jump Smooth RHS	Trivial jump Discont. RHS	Crown jump Smooth RHS	Crown jump Discont. RHS
$p = 1$		2.04	1.61	1.54
$p = 2$		2.39	2.33	2.32
$p = 3$	4.07	2.36	3.63	3.62
$p = 4$	4.56	2.46	4.03	4.03

TABLE 3.2
Convergence rates from numerical experiments, in the sip+-norm.

DG degree	Trivial jump Smooth RHS	Trivial jump Discont. RHS	Crown jump Smooth RHS	Crown jump Discont. RHS
$p = 1$		1.36	1.32	1.31
$p = 2$		1.63	2.08	2.08
$p = 3$	3.16	1.52	2.11	2.11
$p = 4$	3.69	1.56	2.38	2.38

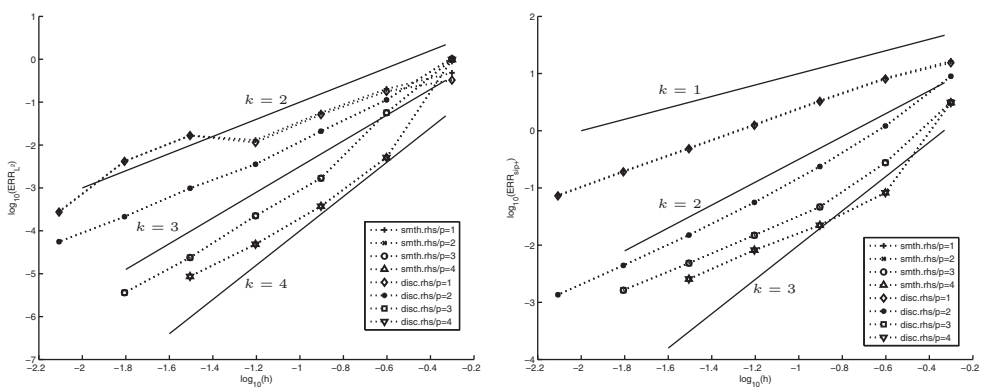

FIG. 3.5. *Convergence plot for crown jump. It is shown that the RHS has only a minor effect on the convergence, since for any given polynomial order the plots for smooth and discontinuous RHSs are practically identical.*

- From Figure 3.6 it is obvious that the approximation procedure for the ansatz function yields a convergence order of $p+1$ in the $L^2(\mathfrak{I})$ order. There is hardly any influence from the quadrature rule that is used for computing the ansatz function and the DG polynomial order of the level set representation, as can be seen in Figure 3.7.
- Also the RHS seems to have almost no influence, since the errors for smooth and discontinuous RHS in Figure 3.5 almost match for each polynomial degree, respectively. Since the approximation order of the level set function φ is the same for all examples, namely, $\varphi \in \mathbb{P}_5(\mathfrak{K})$, we reason that it is not the precision of the level set representation that dominates the error. Otherwise, we would expect that the influence of the RHS would vary among different polynomial degrees for u. Also Figure 3.7 indicates that the representation of the level set is of minor influence.

Three-dimensional example calculation, comments on performance. Given is a domain $\Omega = (-2, 2)^3$ discretized by an unstructured tetrahedral grid with 10,118 cells; the cell diameter is between 0.189 and 0.765 (see Figure 3.8). The polynomial

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

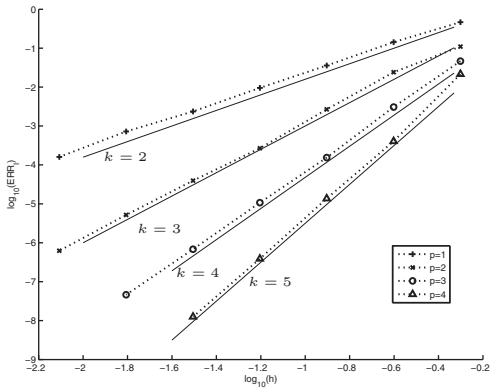

FIG. 3.6. *Convergence for the jump itself, in the L^2-norm on the interface.*

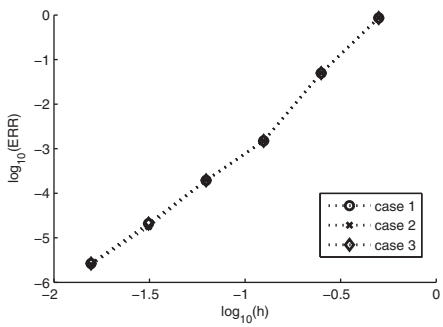

FIG. 3.7. *Influence on solution accuracy from quadrature rule for the ansatz function and level set representation: case* 1 *represents the standard case, the RHS is smooth* ($\mathcal{C}^\infty$) *and the crown jump is used. The DG polynomial degree is* 3*. For case* 2 *the quadrature precision for computing the ansatz is enhanced; each cell is split into* 4^2 *subcells, and within each subcell a Gauss rule of order* 15 *is used, instead of the* 7*th-order rule without subdivisions from the previous case. In case* 3*, the polynomial degree of the level set is increased from* 5 *to* 7*, and a Gauss quadrature rule of order* 29 *is used (instead of an* 11*th-order rule) to project the function* $\varphi(\boldsymbol{x}) = 1.3 - |\boldsymbol{x}|_2$ *onto* $DG_7(\mathfrak{K})$*. Clearly visible, all error plots almost match.*

degree of the DG interpolation is 2. The initial level set function is given as

$$\begin{aligned}\varphi^1(x,y,z) := &-2.985984|x|^6 + 14.40|x|^2 + 10 \\ &-15.38506625|y^2(0.5x+1.5)^2 + z^2(-0.5x+1.5)^2|^{0.55}\end{aligned}$$

and the procedure from [11] was utilized in order to find some $\varphi \in \mathbb{P}_4(\mathfrak{K})$ which is approximately signed distance and has approximately the same zero set as φ^1. 919 cells are cut by the interface. The jump conditions are given as

$$\begin{aligned}[\![u]\!] &= 1 - 0.1xy &&=: g_1,\\ [\![\boldsymbol{n}_\mathfrak{I} \cdot \nabla u]\!] &= x &&=: g_2.\end{aligned}$$

On $\partial\Omega$, homogeneous Dirichlet boundary conditions are assumed; the RHS of the Poisson problem is given as

$$f := 5\exp(-(x^2+y^2)).$$

Numerical results are shown in Figure 3.8.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 04/20/26 to 94.72.173.234 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

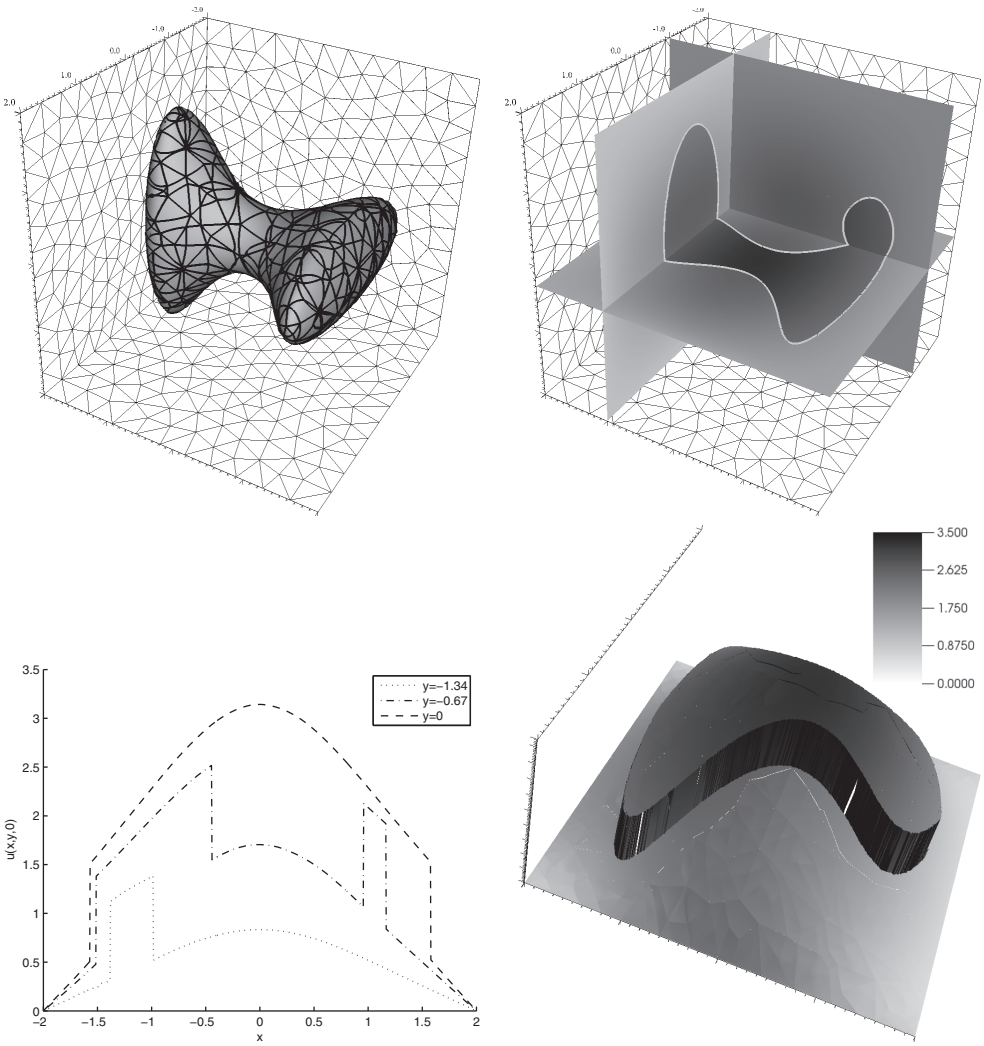

FIG. 3.8. *Three-dimensional Poisson problem with jump. Top left: the interface and the unstructured gird (*10,118 *cells with diameter between* 0.189 *and* 0.765*). Top right: solution contours (dotted) and interface* $\mathfrak{I}$ *(thick) at cross sections* $x = -1$*,* $y = -0.3$*, and* $z = -1$*. Bottom left: solution profile along x-axis, at* $y = -1.34$*,* $y = -0.67$*,* $y = 0$*, and* $z = 0$*. Bottom right: the solution* u *in the* $y = 0$*-plane.*

Performance: The presented XDG method may be most advantageous in the case of moving interfaces, i.e., if the Poisson problem is solved multiple times with a different interface position at each time. In this case the matrix of the system is not altered and no remeshing is necessary when the interface position is changed. This is in strong contrast to traditional schemes, where the grid has to be remeshed and is adapted to the interface position every time as it changes.

A conjugate gradient (CG) algorithm (see, e.g., [23]), with incomplete LU decomposition (ILU(4), implementation is the Euclid preconditioner from the HYPRE package, [14, 15]), was used for solving the linear system. The termination criterion is set to an absolute residual 2-norm of 10^{-12}, i.e., the algorithm terminates if $\|M \cdot x - b\|_2 < 10^{-12}$ for the linear system $M \cdot x = b$. The whole computation utilizes only one CPU core (AMD Opteron 2350 2 GHz).

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 04/20/26 to 94.72.173.234 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Downloaded 04/20/26 to 94.72.173.234 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

The overhead, in terms of CPU time, can be broken down into five individual parts:

1. The computation of the ansatz function $\mathbf{1}_{K_C} u_A$ within the patching domain. In this example, it took 2.77 seconds to compute (approximately 3.3% of the total runtime).
2. The computation of the patching-correction vector $c_+^*[-]$. Runtime is 1.17 seconds (approximately 1.4% of the total runtime). It should be noted that our implementation contains a reasonable amount of software optimization potential in comparison to the rest of the code.
3. The solution of the linear system. Runtime is 29.61 seconds (approximately 70% of the total runtime) for 46 iterations. Of course, this strongly depends on preconditioning.
4. The DG projection of the RHS of the Poisson equation, i.e., the term
$$Proj_p\left(\sum_{\mathcal{S}\in\{\mathfrak{A},\mathfrak{B}\}} L_\Delta(Cont_{\mathcal{S}}(u_A))\mathbf{1}_{\mathcal{S}}\right)$$
(see (3.7) and (3.5)) requires 29.8 seconds to compute, which is about 35.8% of the overall runtime. Since the integrand is not smooth within the cut cells, a very precise quadrature rule must be used. We use an adaptive strategy presented by Müller et al. in [20]. The tetrahedral cells are—adaptively—subdivided up to 5 times, and a Gauss rule of order 4 is used in each subdivision.
5. The runtime of the matrix assembly—which is exactly the same for both the discontinuous and the smooth problems—is approximately 19.84 seconds (approximately 24% of the total runtime). This, however may heavily depend on the problem dimension (two or three), DG polynomial degree, and many other factors. The "cost" of computing $c_+^*[-]$ seems to be quite small in comparison to matrix assembly and negligible in comparison to the solver runtime.

4. An iterative procedure for linear-affine jump conditions. Up to here, only affine jump conditions in the form $[\![u]\!] = g$ have been investigated. It is clear that the solution is a member of an affine linear manifold within the XDG space, i.e., $u \in u_A + \mathbb{P}_p(\mathfrak{K})$, for an ansatz function u_A. More complicated jump conditions would be affine linear ones such as
$$\alpha u_{\mathfrak{A}} + \beta u_{\mathfrak{B}} = \gamma,$$
with $\alpha \neq \beta$, $\alpha, \beta \neq 0$. Here, $u_{\mathfrak{A}}$ and $u_{\mathfrak{B}}$ denote the limit of u on the interface, when approaching from domain $\mathfrak{A}$ or $\mathfrak{B}$, respectively. First numerical experiments have shown that such jump conditions could be approximated in an iterative procedure as follows. Given is a problem which formally reads as
$$\left\{\begin{array}{rl} \mathcal{L}\, u = f & \text{in } \Omega\setminus\mathfrak{I}, \\ \beta u_{\mathfrak{B}} + \alpha u_{\mathfrak{A}} = \gamma & \text{on } \mathfrak{I}, \end{array}\right. \tag{4.1}$$
with a linear operator $\mathcal{L}$. This problem may be substituted for by *a sequence* of problems with affine jump conditions
$$\left\{\begin{array}{rl} \mathcal{L}\, v^n = f & \text{in } \Omega\setminus\mathfrak{I}, \\ [\![v^n]\!] = c^n & \text{on } \mathfrak{I}, \end{array}\right. \qquad n = 1, 2, 3, \ldots, \tag{4.2}$$

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 04/20/26 to 94.72.173.234 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

which can be solved in the way that we propose for the Poisson equation (see section 3) and which converges to the solution of the original problem (4.1), i.e.,

$$\left\{ \begin{array}{r} v^n \to u, \\ \beta v^n_{\mathfrak{B}} + \alpha v^n_{\mathfrak{A}} \to \gamma \end{array} \right. \quad \text{for } n \to \infty. \tag{4.3}$$

For the iteration process, we define a "biased center"

$$v_c^{n-1} = v_{\mathfrak{A}}^{n-1}(1-\vartheta) + v_{\mathfrak{B}}^{n-1}\vartheta$$

of the jump of v^{n-1}, with a bias constant $\vartheta \in (0,1)$. It is assumed that the jump of the unknown v^n will also be centered around v_c^{n-1}, i.e.,

$$v^n_{\mathfrak{A}} \approx v_c^{n-1} - c^n\vartheta \quad \text{and} \quad v^n_{\mathfrak{B}} \approx v_c^{n-1} + c^n(1-\vartheta).$$

By assuming these approximations to be exact and inserting them into the linear-affine jump condition of problem (4.1), one gains the expression

$$c^n = \frac{\gamma + (\alpha+\beta)v_c^{n-1}}{(1-\vartheta)\beta - \vartheta\alpha}.$$

If the iteration is convergent, i.e., $v^n \to u$, v_c^n and c^n are also convergent. In numerical experiments, a relaxation was necessary in order to observe convergence, i.e.,

$$[\![v^n]\!] = c^n\varepsilon_{rlx} + [\![v^{n-1}]\!]\,(1-\varepsilon_{rlx})$$

with a relaxation constant $\varepsilon_{rlx} \in (0,1)$.

Numerical experiment. On the domain $\Omega = (-1,1)$ with $\mathfrak{A} = (0,1)$, $\mathfrak{B} = (-1,0)$, and $\mathfrak{I} = \{0\}$, the equation

$$\left\{ \begin{array}{rl} -\Delta u = 1/\mu & \text{in } \Omega \setminus \mathfrak{I}, \\ [\![u]\!] = 0 & \text{on } \mathfrak{I}, \\ [\![\mu \frac{\partial}{\partial x} u]\!] = 0 & \text{on } \mathfrak{I}, \\ u = 0 & \text{on } \partial\Omega \end{array} \right.$$

with $\mu = \left\{ \begin{array}{ll} \mu_{\mathfrak{A}} = 17.1\cdot 10^{-6} & \text{in } \mathfrak{A} \\ \mu_{\mathfrak{B}} = 10^{-3} & \text{in } \mathfrak{B} \end{array} \right.$ is given. An equidistant grid with 21 cells was used; the DG polynomial order is 2. The exact solution is

$$u(x) = \left\{ \begin{array}{ll} a_0 + a_1 x + a_2 x^2 & \text{in } \mathfrak{A}, \\ b_0 + b_1 x + b_2 x^2 & \text{in } \mathfrak{B}, \end{array} \right.$$

with coefficients

$$a_0 = \frac{1}{\mu_{\mathfrak{A}} + \mu_{\mathfrak{B}}}, \quad a_1 = \frac{\mu_{\mathfrak{B}} - \mu_{\mathfrak{A}}}{2(\mu_{\mathfrak{B}} + \mu_{\mathfrak{A}})\mu_{\mathfrak{A}}}, \quad a_2 = \frac{-1}{2\mu_{\mathfrak{A}}},$$
$$b_0 = \frac{1}{\mu_{\mathfrak{A}} + \mu_{\mathfrak{B}}}, \quad b_1 = \frac{\mu_{\mathfrak{B}} - \mu_{\mathfrak{A}}}{2(\mu_{\mathfrak{B}} + \mu_{\mathfrak{A}})\mu_{\mathfrak{B}}}, \quad b_2 = \frac{-1}{2\mu_{\mathfrak{B}}}.$$

Since $u \in XDG_2(-x, \mathfrak{K})$, the method is expected to converge to the exact solution. As a start for the iteration we chose $u^0 = 0$ and use, for iteration $n > 0$, the jump conditions

$$[\![u^n]\!] = 0 \quad \text{and} \quad [\![\mu \frac{\partial}{\partial x} u^n]\!] = \frac{1}{2}\left(c^n + [\![\mu \frac{\partial}{\partial x} u^{n-1}]\!]\right).$$

Steps of the iteration process are sketched in Figure 4.1.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

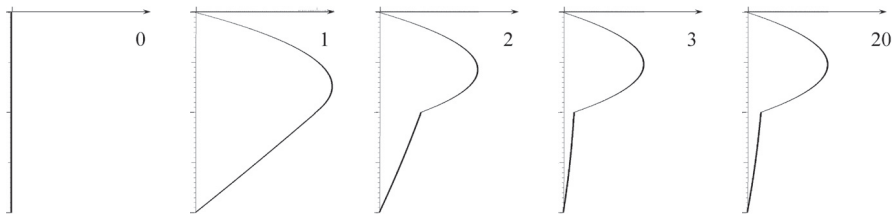

FIG. 4.1. *Some members of the iteration series* u^n *for* $n \in \{0, 1, 2, 3, 20\}$.

5. Summary and outlook. A scheme for the solution of the singular Poisson equation has been developed. Under optimal conditions, the method shows a convergence order of $p + 1$ for DG polynomial degree p. In the worst case, a convergence order of ≈ 2.4 could still be preserved for DG polynomial degree 2.

A very useful property of the proposed method is that the matrix of the linear system is equal to those that one would have to solve for the corresponding problem without any jump. Furthermore, the matrix is independent of the interface position and therefore, for unsteady problems, where the Poisson equation has to be solved multiple times with different interface positions, the (usually expensive) matrix assembly is only required at the very beginning. This property may also be beneficial for preconditioners.

The algorithm presented within this work may be used as one building block of a fractional-step [1] typed multiphase Navier–Stokes solver, like the SIMPLE algorithm [16, 22] or the projection method [10]. In such an environment, the patched-decomposition approach seems to be very promising, since it is capable of handling the interface movement easily and efficient, with very low overhead in comparison to a single-phase solver.

The crucial ingredient seems to be how the jump and kink conditions, which can be derived from conservation laws for mass and momentum and additional constitutive laws, can be integrated into the fractional-step schemes, and the numerical stability of such schemes. This will be the issue of future works.

Acknowledgments. We further have to thank our colleagues Yongqi Wang, Jan-Niklas Hau, Björn Müller, and Benedikt Klein for fruitful discussions on the topic.

REFERENCES

[1] S. ARMFIELD AND R. STREET, *An analysis and comparison of the time accuracy of fractional-step methods for the Navier-Stokes equations on staggered grids*, Internat. J. Numer. Methods Fluids, 38 (2002), pp. 255–282.

[2] D. N. ARNOLD, F. BREZZI, B. COCKBURN, AND L. D. MARINI, *Unified analysis of discontinuous Galerkin methods for elliptic problems*, SIAM J. Numer. Anal., 39 (2002), pp. 1749–1779.

[3] D. N. ARNOLD, *An interior penalty finite element method with discontinuous elements*, SIAM J. Numer. Anal., 19 (1982), pp. 742–760.

[4] P. BASTIAN AND C. ENGWER, *An unfitted finite element method using discontinuous Galerkin*, Internat. J. Numer. Methods Engrg., 79 (2009), pp. 1557–1576.

[5] T. CHEN, P. D. MINEV, AND K. NANDAKUMAR, *A projection scheme for incompressible multiphase flow using adaptive Eulerian grid*, International J. Numer. Methods Fluids, 45 (2004), pp. 1–19.

[6] D. A. DI PIETRO AND A. ERN, *PDEs with diffusion*, in Mathematical Aspects of Discontinuous Galerkin Methods Mathématiques et Applications 69, Springer, Berlin, 2011, pp. 119–186.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 04/20/26 to 94.72.173.234 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

[7] C. Engwer and F. Heimann, *Dune-udg: A cut-cell framework for unfitted discontinuous Galerkin methods*, in Advances in DUNE, A. Dedner, B. Flemisch, and R. Köfkorn, eds., Springer, Berlin, 2012, pp. 89–100.

[8] C. Engwer, *An Unfitted Discontinuous Galerkin Scheme for Micro-scale Simulations and Numerical Upscaling*, Ph.D. thesis, Heidelberg University, Heidelberg, Germany, 2009.

[9] P. Esser, J. Grande, and A. Reusken, *An extended finite element method applied to levitated droplet problems*, Internat. J. Numer. Methods Engrg., 84 (2010), pp. 757–773.

[10] W. E and J.-G. Liu, *Projection method* I: *Convergence and numerical boundary layers*, SIAM J. Numer. Anal., 32 (1995), pp. 1017–1057.

[11] J. Grooss and J. Hesthaven, *A level set discontinuous Galerkin method for free surface flows*, Comput. Methods Appl. Mech. Engrg., 195 (2006), pp. 3406–3429.

[12] S. Gross and A. Reusken, *An extended pressure finite element space for two-phase incompressible flows with surface tension*, J. Comput. Phys., 224 (2007), pp. 40–58.

[13] K. Hutter and K. D. Jöhnk, *Continuum Methods of Physical Modeling: Continuum Mechanics, Dimensional Analysis, Turbulence*, Springer, Berlin, 2004.

[14] D. Hysom and A. Pothen, *Efficient parallel computation of ILU(k) preconditioners*, in Proceedings of the 1999 ACM/IEEE Conference on Supercomputing, CD-ROM, ACM, New York, 1999.

[15] D. Hysom and A. Pothen, *A scalable parallel algorithm for incomplete factor preconditioning*, SIAM J. Sci. Comput., 22 (2000), pp. 2194–2215.

[16] B. Klein, F. Kummer, and M. Oberlack, *A SIMPLE based discontinuous Galerkin solver for steady incompressible flows*, J. Comput. Phys., 237 (2013), pp. 235–250.

[17] E. Marchandise, P. Geuzaine, N. Chevaugeon, and J.-F. Remacle, *A stabilized finite element method using a discontinuous level set approach for the computation of bubble dynamics*, J. Comput. Phys., 225 (2007), pp. 949–974.

[18] E. Marchandise and J.-F. Remacle, *A stabilized finite element method using a discontinuous level set approach for solving two phase incompressible flows*, J. Comput. Phys., 219 (2006), pp. 780–800.

[19] N. Moës, J. Dolbow, and T. Belytschko, *A finite element method for crack growth without remeshing*, Internat. J. Numer. Methods Engrg., 46 (1999), pp. 131–150.

[20] B. Müller, F. Kummer, M. Oberlack, and Y. Wang, *Simple multidimensional integration of discontinuous functions with application to level set methods*, Internat. J. Numer. Methods Engrg., 92 (2012), pp. 637–651.

[21] S. J. Osher and R. P. Fedkiw, *Level Set Methods and Dynamic Implicit Surfaces*, Springer, New York, 2002.

[22] S. V. Patankar and D. B. Spalding, *A calculation procedure for heat, mass and momentum transfer in three-dimensional parabolic flows*, Internat. J. Heat Mass Transfer, 15 (1972), pp. 1787–1972.

[23] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed., SIAM, Philadelphia, 2003.

[24] J. A. Sethian and P. Smereka, *Level set methods for fluid interfaces*, Annu. Rev. Fluid Mech., 35 (2003), pp. 341–372.

[25] J. A. Sethian, *Level Set Methods: Evolving Interfaces in Geometry, Fluid Mechanics, Computer Vision, and Materials Science*, Cambridge University Press, Cambridge, 1996.

[26] J. A. Sethian, *Level Set Methods and Fast Marching Methods*, Cambridge University Press, Cambridge, 1999.

[27] J. A. Sethian, *Evolution, implementation, and application of level set and fast marching methods for advancing fronts*, J. Comput. Phys., 169 (2001), pp. 503–555.

[28] K. Shahbazi, *An explicit expression for the penalty parameter of the interior penalty method*, J. Comput. Phys., 205 (2005), pp. 401–407.

[29] F. van der Bos and V. Gravemeier, *Numerical simulation of premixed combustion using an enriched finite element method*, J. Comput. Phys., 228 (2009), pp. 3605–3624.

[30] Y. Wang and M. Oberlack, *A thermodynamic model of multiphase flows with moving interfaces and contact line*, Contin. Mech. Thermodyn., 23 (2011), pp. 409–433.

Downloaded 04/20/26 to 94.72.173.234 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.